

ЕТАП 0 ОТ 11 · РЪКОВОДСТВО

Основа и главен екран

الأذان

Ехро проект с TypeScript и екрана „Днес“ по одобрения моцкир. Всеки файл е обяснен в реда, в който го създаваш. Кодът е същият като в архива ezan.zip.

Expo SDK 57

React Native 0.86

TypeScript

Expo Router

34 файла



Решения от дизайна, заложиени в кода

Арабски имена на молитвите · Изгрев само с нотификация, без езан · градиентът на текущата молитва е на целия екран · език според телефона: български или английски · по подразбиране: София, метод Диянет, Аср по Шафии.

СЪДЪРЖАНИЕ

01 Какво ти трябва

02 Създаване на проекта

03 Конфигурация

04 Структура на папките

05 Тема: цветове, градиенти, шрифтове

06 Логика: часове, дати, форматиране

07 Състояние и hooks

08 Езици: български и английски

09 Графика: фон, шарка, пръстен

10 Компоненти на екрана „Днес“

11 Навигация и екрани

12 Стартиране на телефона

13 Проверка

Стъпка 1

Какво ти трябва

Преди да започнеш, провери, че на компютъра имаш:

- **Node.js LTS** (20 или по-нов) и **npm**. Провери с `node -v`.
- **Git**.
- **Android Studio** – за Android SDK и емулатор. Не е нужно, ако build-ваш в облака с EAS и инсталираш APK директно на телефона.
- **Android телефон** с включен USB debugging (Настройки → Информация за телефона → 7 пъти „Номер на компилация“ → Опции за разработчици).
- **Акаунт в expo.dev** (безплатен) – за облачните build-ове с EAS.
- За iOS по-късно: Apple Developer акаунт (\$99/год). Мас не е задължителен, защото EAS build-ва в облака.

Имаш два начина да минеш през ръководството: да разархивираш готовия проект `ezan.zip` и да четеш обясненията, или да го изградиш сам файл по файл. Кодът по-долу е точно същият като в архива.

Стъпка 2

Създаване на проекта

Започваме от празния TypeScript шаблон на Expo. Така проектът съдържа само това, което добавим, без примерни екрани.

Нов проект с Expo SDK 57, React Native 0.86 и TypeScript

```
npx create-expo-app@latest ezan --template blank-typescript
cd ezan
```

Добавяме Expo Router и графичните библиотеки. `npx expo install` (а не `npm install`) избира версии, съвместими с твоята версия на Expo SDK.

Библиотеки с native код – винаги през expo install

```
npx expo install expo-router react-native-safe-area-context react-native-screens \
expo-linking expo-constants expo-status-bar \
react-native-svg expo-linear-gradient expo-font expo-splash-screen \
react-native-reanimated react-native-worklets \
@react-native-async-storage/async-storage expo-dev-client \
expo-localization expo-system-ui
```

Чист JavaScript – обикновен npm install

```
npm install adhan zustand @expo-google-fonts/manrope @expo-google-fonts/amiri
```

Настройки за native build-a (версия на CMake за Windows)

```
npx expo install expo-build-properties
```

По желание: бърз преглед в браузъра с `npx expo start -web`

```
npx expo install react-dom react-native-web @expo/metro-runtime
```

ПАКЕТ	ЗА КАКВО Е
expo-router	Навигация чрез файлове: всеки файл в <code>src/app</code> е екран
react-native-svg	Пръстенът, иконите, шарката и силуетът на джамията
expo-linear-gradient	Градиентът на фона според молитвата
react-native-reanimated	Плавните анимации (смяна на фона, toast)
expo-font + @expo-google-fonts/*	Шрифтовете Manrope и Amiri, вградени в приложението (работят офлайн)
adhan	Изчисляване на часовете за намаз по координати, без интернет
zustand + async-storage	Малък store за режимите на камбанките, който се запазва между стартиранията
expo-dev-client	Прави твоя собствен development build (вместо Expo Go)
expo-localization	Езикът на телефона: български или английски
expo-system-ui	Тъмен фон на самия прозорец (без бял проблясък при стартиране)
expo-build-properties	Настройки на native build-a: версия на CMake (важно за Windows)

Изтрий `App.tsx` и `index.ts` от шаблона. С Expo Router входната точка е самият router.

Терминал

```
rm App.tsx index.ts
mkdir -p "src/app/(tabs)" src/components src/config src/domain src/hooks src/store src/theme
```

Стъпка 3

Конфигурация

Три конфигурационни файла свързват всичко: къде е входната точка, как се казва приложението и как TypeScript намира файловете.

package.json корен на проекта

- `main` сочи към `expo-router/entry`. Router-ът сам намира екраните в `src/app`.
- `android` / `ios` скриптовете вече правят **собствен build** (`expo run`), а не отварят Expo Go.
- `typecheck` и `lint` пускай преди всеки commit.

package.json

49 реда

```
{
  "name": "ezan",
  "version": "1.0.0",
  "main": "expo-router/entry",
  "dependencies": {
    "@expo-google-fonts/amiri": "^0.4.1",
```

```

"@expo-google-fonts/manrope": "^0.4.2",
"@expo/metro-runtime": "~57.0.16",
"@react-native-async-storage/async-storage": "2.2.0",
"adhan": "^4.4.6",
"expo": "~57.0.25",
"expo-build-properties": "~57.0.22",
"expo-constants": "~57.0.19",
"expo-dev-client": "~57.0.19",
"expo-font": "~57.0.4",
"expo-linear-gradient": "~57.0.2",
"expo-linking": "~57.0.11",
"expo-localization": "~57.0.2",
"expo-router": "~57.0.23",
"expo-splash-screen": "~57.0.9",
"expo-status-bar": "~57.0.1",
"expo-system-ui": "~57.0.4",
"react": "19.2.3",
"react-dom": "19.2.3",
"react-native": "0.86.3",
"react-native-reanimated": "4.5.1",
"react-native-safe-area-context": "~5.7.0",
"react-native-screens": "~4.26.0",
"react-native-svg": "15.15.4",
"react-native-web": "~0.21.0",
"react-native-worklets": "0.10.1",
"zustand": "^5.0.15"
},
"devDependencies": {
"@types/react": "~19.2.2",
"eslint": "^9.0.0",
"eslint-config-expo": "~57.0.2",
"typescript": "~6.0.3"
},
"scripts": {
"start": "expo start",
"android": "expo run:android",
"ios": "expo run:ios",
"web": "expo start --web",
"typecheck": "tsc --noEmit",
"lint": "expo lint"
},
"private": true
}

```

app.json корен на проекта

- `name` е името под иконата по подразбиране: „Adhan“. `locales` го превежда: на телефон на български се показва „Езан“ (файловете `locales/bg.json` и `locales/en.json`). `scheme` позволява линкове от вида `ezan://` (ще ни трябват за нотификациите).
- `userInterfaceStyle: "dark"` и `backgroundColor` – приложението винаги е тъмно, без бял проблясък при стартиране.
- `com.example.ezan` е временно. Смени го с твоё име (напр. `bg.ilko.ezan`) **преди** първото качване в магазините – после не може да се промени.
- Plugin-ът на splash екрана ползва същото нощно синьо.
- Plugin-ът `expo-localization` със `supportedLocales` обявява двата езика. Така на Android 13+ и iOS потребителят може да избере език само за това приложение от настройките на телефона.

- `expo-build-properties` задава CMake 3.31.6 за native частта. Новият `ninja` в него поддържа дълги пътища в Windows (виж стъпка 12).

app.json

74 реда

```
{
  "expo": {
    "name": "Adhan",
    "slug": "ezan",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "userInterfaceStyle": "dark",
    "ios": {
      "supportsTablet": false,
      "bundleIdentifier": "com.example.ezan",
      "infoPlist": {
        "CFBundleAllowMixedLocalizations": true
      }
    },
    "android": {
      "adaptiveIcon": {
        "backgroundColor": "#0F1B2D",
        "foregroundImage": "./assets/android-icon-foreground.png",
        "backgroundImage": "./assets/android-icon-background.png",
        "monochromeImage": "./assets/android-icon-monochrome.png"
      },
      "predictiveBackGestureEnabled": false,
      "package": "com.example.ezan"
    },
    "web": {
      "favicon": "./assets/favicon.png"
    },
    "plugins": [
      "expo-router",
      [
        "expo-splash-screen",
        {
          "backgroundColor": "#0F1B2D",
          "image": "./assets/splash-icon.png",
          "imageWidth": 160
        }
      ],
      "expo-font",
      [
        "expo-build-properties",
        {
          "android": {
            "cmakeVersion": "3.31.6"
          }
        }
      ],
      [
        "expo-localization",
        {
          "supportedLocales": {
            "ios": [
              "en",
              "bg"
            ],
            "android": [
```

```

        "en",
        "bg"
      ]
    }
  }
],
"scheme": "ezan",
"backgroundColor": "#0F1B2D",
"experiments": {
  "typedRoutes": true
},
"locales": {
  "bg": "./locales/bg.json",
  "en": "./locales/en.json"
}
}
}

```

bg.json locales

- Името под иконата на телефон на български: „Езан“. Файлт `locales/en.json` е същият, но с „Adhan“.
- Това са системни текстове: вграждат се при build, а не в JavaScript. Затова промяна тук изисква нов build.

locales/bg.json

8 реда

```

{
  "ios": {
    "CFBundleDisplayName": "Езан"
  },
  "android": {
    "app_name": "Езан"
  }
}

```

tsconfig.json корен на проекта

- `paths` прави `@/` да сочи към `src/`, затова пишем `import ... from '@theme/colors'` вместо дълги относителни пътища.
- `strict: true` – пълна проверка на типовете.

tsconfig.json

17 реда

```

{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true,
    "paths": {
      "@/*": [
        "./src/*"
      ]
    }
  },
  "include": [
    "**/*.ts",
    "**/*.tsx",
    ".expo/types/**/*.ts",
  ]
}

```

```
"expo-env.d.ts"
]
}
```

eas.json корен на проекта

- Профилите за облачни build-ове. `development` прави APK с dev client, който се свързва с компютъра ти за hot reload.
- `preview` е за тестване от други хора, `production` – за магазините.

eas.json

23 реда

```
{
  "cli": {
    "version": ">= 16.0.0",
    "appVersionSource": "remote"
  },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal",
      "android": { "buildType": "apk" }
    },
    "preview": {
      "distribution": "internal",
      "android": { "buildType": "apk" }
    },
    "production": {
      "autoIncrement": true
    }
  },
  "submit": {
    "production": {}
  }
}
```

Стъпка 4

Структура на папките

Правилото е просто: в `src/app` има само екрани (маршрути). Всичко останало живее извън нея, разделено по роля.

Дърво на проекта

```
src/
├─ app/                маршрути (всеки файл е екран)
│   ├── _layout.tsx    коренът: шрифтове, splash, status bar
│   └── (tabs)/         група с долна навигация (скобите не влизат в URL-a)
│       ├── _layout.tsx табове
│       ├── index.tsx   „Днес“
│       ├── qibla.tsx   „Кибла“ (засега празен)
│       ├── month.tsx   „Месец“ (засега празен)
│       └── settings.tsx „Настройки“ (засега празен)
├─ components/         UI парчета без собствена логика за данни
├─ config/             стойности по подразбиране (град, метод)
├─ domain/             чиста логика без React: часове, дати, форматиране
├─ hooks/              React hooks, които свързват domain с UI
├─ i18n/               текстовете на български и английски
├─ store/              състояние, което се запазва (zustand)
├─ theme/              цветове, градиенти, шрифтове
└─ locales/            името на приложението под иконата (bg / en)
```

`domain/` не импортира нищо от React или React Native. Така е лесна за тестване (етап 2) и може да се преизползва, например в widget-a.

Стъпка 5

Тема: цветове, градиенти, шрифтове

Всички стойности от одобрения `moskip` са на едно място. Компонентите никога не пишат hex код директно.

colors.ts `src/theme`

- Основната палитра плюс полупрозрачните слоеве (картата на списъка, лентата с табове), които стоят върху градиента.
- as const** прави стойностите literal типове, така че TypeScript подсказва точните имена.

`src/theme/colors.ts`

24 реда

```
/**
 * Палитрата от одобрения moskip (v2).
 * Приложението е винаги тъмно, затова няма отделна светла тема.
 */
export const colors = {
  base: '#0F1B2D', // нощно синьо, основа
  gold: '#D4A857', // акцент: следваща молитва, активни елементи
  goldInk: '#1A1408', // тъмен текст върху злато
  text: '#F2EFE8',
  textDim: 'rgba(242,239,232,0.72)',
  muted: '#8A96A8', // вторичен текст
  warn: '#E39A4B',

  // полупрозрачни слоеве върху градиента
  card: 'rgba(6,10,20,0.42)',
  cardBorder: 'rgba(255,255,255,0.08)',
  tabBar: 'rgba(6,10,20,0.62)',
  pill: 'rgba(255,255,255,0.08)',
  bellNotify: 'rgba(255,255,255,0.10)',
  ringTrack: 'rgba(255,255,255,0.12)',
```

```

nowRow: 'rgba(212,168,87,0.15)',
nowRowBorder: 'rgba(212,168,87,0.50)',
skyline: 'rgba(0,0,0,0.22)',
} as const;

```

gradients.ts src/theme

- По три цвята за всяка молитва (горе → среда → долу), същите като в mockup-а.
- `Record<PrayerId, ...>` гарантира, че ако някога добавиш молитва, TypeScript ще те накара да ѝ дадеш градиент.

src/theme/gradients.ts

15 реда

```

import type { PrayerId } from '@domain/prayers';

/** Три цвята за всяка молитва: горе → среда (55%) → долу. */
export type GradientStops = readonly [string, string, string];

export const PHASE_GRADIENTS: Record<PrayerId, GradientStops> = {
  fajr: ['#14132C', '#3A2A57', '#9A6079'], // зора: лилаво → розово
  sunrise: ['#111F35', '#2E4F72', '#C98E62'], // утро: синьо → праскова
  dhuhr: ['#0D2A47', '#1E5A86', '#5B98C0'], // пладне: небесно синьо
  asr: ['#1B1712', '#4F3C1F', '#B48437'], // следобед: кехлибар
  maghrib: ['#1B1023', '#672838', '#CC6139'], // залез: бордо → оранжево
  isha: ['#070A1B', '#131941', '#2A2C66'], // нощ: индиго
};

export const GRADIENT_LOCATIONS = [0, 0.55, 1] as const;

```

typography.ts src/theme

- В React Native всяка дебелина на шрифта е отделно семейство (`Manrope_700Bold`), затова не ползваме `fontWeight`.
- `tabularNums` дава на цифрите еднаква ширина. Без това обратното броене „подскача“ всяка секунда, защото „1“ е по-тясна от „8“.

src/theme/typography.ts

18 реда

```

import type { TextStyle } from 'react-native';

/**
 * Имената съвпадат с ключовете, с които шрифтовете се зареждат в src/app/_layout.tsx.
 * В React Native всяка дебелина е отделно семейство, затова не ползваме fontWeight.
 */
export const fonts = {
  regular: 'Manrope_400Regular',
  medium: 'Manrope_500Medium',
  semibold: 'Manrope_600SemiBold',
  bold: 'Manrope_700Bold',
  extrabold: 'Manrope_800ExtraBold',
  arabic: 'Amiri_400Regular',
  arabicBold: 'Amiri_700Bold',
} as const;

/** Цифри с еднаква ширина, за да не „подскача“ обратното броене. */
export const tabularNums: TextStyle = { fontVariant: ['tabular-nums'] };

```

Логика: часове, дати, форматиране

Това е сърцето на приложението. Всичко тук е чист TypeScript без UI и може да се тества самостоятелно.

defaults.ts src/config

- Докато няма екран за настройки и GPS, приложението ползва София, метод Диянет и Аср по Шафии.
- **Защо Шафии, а не Ханафи:** официалните календари на Диянет (и на Мюфтийството) дават „първия“ Аср – когато сянката е равна на предмета. Това е изчислението, което `adhan` нарича Shafi. Ханафи дава Аср около час по-късно. В москир-а на настройките беше избран Ханафи – ще го сменим в етап 3.
- Точните стойности ще свериш с календара на Мюфтийството в етап 2.

src/config/defaults.ts

28 реда

```
/**
 * Стойности по подразбиране, докато няма екран за настройки и GPS (етапи 2 и 3 от плана).
 */
export type CalculationMethodId = 'Turkey' | 'MuslimWorldLeague' | 'Egyptian' | 'UmmAlQura';
export type AsrMadhab = 'shafi' | 'hanafi';

export interface AppLocation {
  id: string;
  /** Името на града на двата езика (градовете са данни, а не текстове на интерфейса). */
  names: { bg: string; en: string };
  latitude: number;
  longitude: number;
}

export const DEFAULT_LOCATION: AppLocation = {
  id: 'sofia',
  names: { bg: 'София', en: 'Sofia' },
  latitude: 42.6977,
  longitude: 23.3219,
};

/**
 * Диянет (Турция) е най-близо до практиката в България. Сверете с календара на Мюфтийството (етап 2).
 * Официалните календари на Диянет ползват „първия“ Аср (сянка x1), т.е. изчислението по Шафии,
 * затова това е подразбиращата се стойност. Ханафи (сянка x2) дава Аср около час по-късно.
 */
export const DEFAULT_METHOD: CalculationMethodId = 'Turkey';
export const DEFAULT_MADHAB: AsrMadhab = 'shafi';
```

prayers.ts src/domain

- `PRAYER_IDS` задава реда, `PRAYERS` – арабското изписване и дали молитвата може да има езан. Изгрев е с `canAlarm: false` (решение №2).
- `computeDay` вика `adhan` за един ден. Резултатът са `Date` обекти в абсолютно време, които се показват в часовата зона на телефона.
- `computeThreeDays` смята вчера, днес и утре. Нужни са за двата гранични случая: **преди Фаджр** текущата молитва е вчерашната Иша, а **след Иша** следващата е утрешният Фаджр.

- Датата се създава в 12:00 на обяд. Така смяната на лятно/зимно време в 3–4 ч. сутринта не може да я измести в съседен ден.
- `getSchedule` връща всичко, което екранът показва: текуща, следваща, прогрес за пръстена, оставащо време и състоянието на всеки ред (минала / сега / предстояща).

src/domain/prayers.ts

103 реда

```
import { CalculationMethod, Coordinates, Madhab, PrayerTimes } from 'adhan';

import type { AppLocation, AsrMadhab, CalculationMethodId } from '@config/defaults';

/** Редът е важен: така се показват в списъка и така се търси „следваща“. */
export const PRAYER_IDS = ['fajr', 'sunrise', 'dhuhr', 'asr', 'maghrib', 'isha'] as const;
export type PrayerId = (typeof PRAYER_IDS)[number];

/** Имената на кирилица/латиница са в src/i18n/strings.ts (зависят от езика). */
export interface PrayerMeta {
  id: PrayerId;
  arabic: string;
  /** Изгрев е само нотификация, без езан/аларма (решение №2). */
  canAlarm: boolean;
}

export const PRAYERS: Record<PrayerId, PrayerMeta> = {
  fajr: { id: 'fajr', arabic: 'الفجر', canAlarm: true },
  sunrise: { id: 'sunrise', arabic: 'الشروق', canAlarm: false },
  dhuhr: { id: 'dhuhr', arabic: 'الظهر', canAlarm: true },
  asr: { id: 'asr', arabic: 'العصر', canAlarm: true },
  maghrib: { id: 'maghrib', arabic: 'المغرب', canAlarm: true },
  isha: { id: 'isha', arabic: 'العشاء', canAlarm: true },
};

export interface PrayerTime {
  id: PrayerId;
  time: Date;
}

export interface CalcOptions {
  location: AppLocation;
  method: CalculationMethodId;
  madhab: AsrMadhab;
}

/** Часовете за един календарен ден. Изчисляват се онлайн по координати. */
export function computeDay(date: Date, opts: CalcOptions): PrayerTime[] {
  const params = CalculationMethod[opts.method]();
  params.madhab = opts.madhab === 'hanafi' ? Madhab.Hanafi : Madhab.Shafi;
  const coords = new Coordinates(opts.location.latitude, opts.location.longitude);
  const pt = new PrayerTimes(coords, date, params);
  return PRAYER_IDS.map((id) => ({ id, time: pt[id] }));
}

export interface ThreeDays {
  yesterday: PrayerTime[];
  today: PrayerTime[];
  tomorrow: PrayerTime[];
}

export function computeThreeDays(now: Date, opts: CalcOptions): ThreeDays {
  const day = (offset: number) =>
```

```

    new Date(now.getFullYear(), now.getMonth(), now.getDate() + offset, 12);
  return {
    yesterday: computeDay(day(-1), opts),
    today: computeDay(day(0), opts),
    tomorrow: computeDay(day(1), opts),
  };
}

export type RowState = 'past' | 'now' | 'upcoming';

export interface Schedule {
  today: PrayerTime[];
  /** Текущата молитва. Преди Фаджр това е Иша от вчера. */
  current: PrayerTime;
  /** Следващата молитва. След Иша това е Фаджр от утре. */
  next: PrayerTime;
  isNextTomorrow: boolean;
  /** 0..1 – колко от интервала current → next е изминал (за пръстена). */
  progress: number;
  remainingMs: number;
  rowState: (id: PrayerId) => RowState;
}

export function getSchedule(now: Date, days: ThreeDays): Schedule {
  const t = now.getTime();
  const { today } = days;

  const passed = today.filter((p) => p.time.getTime() <= t);
  const current = passed.length > 0 ? passed[passed.length - 1] : days.yesterday[5];

  const upcoming = today.find((p) => p.time.getTime() > t);
  const next = upcoming ?? days.tomorrow[0];

  const span = next.time.getTime() - current.time.getTime();
  const progress = span > 0 ? Math.min(1, Math.max(0, (t - current.time.getTime()) / span)) : 0;

  return {
    today,
    current,
    next,
    isNextTomorrow: !upcoming,
    progress,
    remainingMs: Math.max(0, next.time.getTime() - t),
    rowState: (id) => {
      if (id === current.id) return 'now';
      const row = today.find((p) => p.id === id)!;
      return row.time.getTime() <= t ? 'past' : 'upcoming';
    },
  };
}

```

hijri.ts src/domain

- Първо се пробва календарът Umm al-Qura през Intl. Ако телефонът не го поддържа, се ползва табличният алгоритъм – чиста аритметика, винаги работи.
- Проверката на годината (1400–1600) хваща случая, в който някой телефон тихо връща григорианска дата.

- Имената на месеците по Хиджра се подават отвън (`formatHijri(date, t.hijriMonths)`), затова файлът не зависи от езика.
- Датата по Хиджра зависи от наблюдението на луната, затова може да се разминава с 1-2 дни. `adjustDays` ще стане настройка в етап 2.

src/domain/hijri.ts

83 реда

```
/**
 * Дата по Хиджра.
 * 1) Първо опитва календара Umm al-Qura през Intl (ако телефонът го поддържа).
 * 2) Иначе ползва табличния („кувейтски“) алгоритъм, който е чисто аритметичен.
 * И двата могат да се разминат с 1-2 дни с датата, обявена от Мюфтийството
 * (тя зависи от наблюдението на луната). Затова има `adjustDays` -
 * в етап 2 става настройка „Корекция на датата по Хиджра“.
 * Имената на месеците са в src/i18n/strings.ts (hijriMonths).
 */

export interface HijriDate {
  day: number;
  month: number; // 1..12
  year: number;
}

function julianDayNumber(year: number, month: number, day: number): number {
  const a = Math.floor((14 - month) / 12);
  const y = year + 4800 - a;
  const m = month + 12 * a - 3;
  return (
    day +
    Math.floor((153 * m + 2) / 5) +
    365 * y +
    Math.floor(y / 4) -
    Math.floor(y / 100) +
    Math.floor(y / 400) -
    32045
  );
}

function fromIntl(date: Date): HijriDate | null {
  try {
    const parts = new Intl.DateTimeFormat('en-u-ca-islamic-umalqura', {
      day: 'numeric',
      month: 'numeric',
      year: 'numeric',
    }).formatToParts(date);
    const num = (type: string) => parseInt(parts.find((p) => p.type === type)?.value ?? '', 10);
    const day = num('day');
    const month = num('month');
    const year = num('year');
    // проверка, че наистина е върнат ислямски календар, а не григориански
    if (!(day >= 1 && day <= 30 && month >= 1 && month <= 12 && year > 1400 && year < 1600)) {
      return null;
    }
    return { day, month, year };
  } catch {
    return null;
  }
}

function tabular(date: Date): HijriDate {
```

```

const jd = julianDayNumber(date.getFullYear(), date.getMonth() + 1, date.getDate());

let l = jd - 1948440 + 10632;
const n = Math.floor((l - 1) / 10631);
l = l - 10631 * n + 354;
const j =
  Math.floor((10985 - l) / 5316) * Math.floor((50 * l) / 17719) +
  Math.floor(l / 5670) * Math.floor((43 * l) / 15238);
l =
  l -
  Math.floor((30 - j) / 15) * Math.floor((17719 * j) / 50) -
  Math.floor(j / 16) * Math.floor((15238 * j) / 43) +
  29;
const month = Math.floor((24 * l) / 709);
const day = l - Math.floor((709 * month) / 24);
const year = 30 * n + j - 30;

return { day, month, year };
}

export function toHijri(date: Date, adjustDays = 0): HijriDate {
  const d = new Date(date.getFullYear(), date.getMonth(), date.getDate() + adjustDays, 12);
  return fromIntl(d) ?? tabular(d);
}

/** „17 Ребиул-ахир 1448“ или „17 Rabi al-Thani 1448“ */
export function formatHijri(date: Date, monthNames: readonly string[], adjustDays = 0): string {
  const h = toHijri(date, adjustDays);
  return `${h.day} ${monthNames[h.month - 1]} ${h.year}`;
}

```

format.ts src/domain

- Имената на дните и месеците идват като параметър от `i18n`, а не от `Intl`. `Intl` в React Native (Hermes) зависи от телефона и може да върне различен текст на различни устройства.
- Часът е в 24-часов формат и на двата езика.
- `formatCountdown` закръгля нагоре (`ceil`), за да не покаже 00:00:00 една секунда преди времето.

src/domain/format.ts

29 реда

```

/**
 * Форматиране без Intl: имената на дни и месеци идват от src/i18n/strings.ts,
 * за да изглеждат еднакво на всички телефони (Intl в Hermes зависи от устройството).
 */
export interface DateNames {
  weekdaysShort: readonly string[]; // 0 = неделя
  monthsShort: readonly string[];
}

const pad = (n: number) => String(n).padStart(2, '0');

/** 19:13 - 24-часов формат и на двата езика. */
export function formatHM(date: Date): string {
  return `${pad(date.getHours())}:${pad(date.getMinutes())}`;
}

/** 01:49:12 */
export function formatCountdown(ms: number): string {
  const total = Math.max(0, Math.ceil(ms / 1000));

```

```

const h = Math.floor(total / 3600);
const m = Math.floor((total % 3600) / 60);
const s = total % 60;
return `${pad(h)}:${pad(m)}:${pad(s)}`;
}

/** „Пн, 28 септ.“ или „Mon, 28 Sep“ */
export function formatGregorianShort(date: Date, names: DateNames): string {
  return `${names.weekdaysShort[date.getDay()]}, ${date.getDate()} ${names.monthsShort[date.getMonth()]}`;
}

```

Стъпка 7

Състояние и hooks

Тук свързваме логиката с React: колко е часът, кои са днешните часове и какво е избрал потребителят за всяка камбанка.

alertPrefs.ts src/store

- Zustand store с режима за всяка молитва: `off` / `notify` / `adhan`.
- `nextMode` върти цикъла и прескача „езан“ за Изгрев.
- `persist` с AsyncStorage запазва избора между стартиранията. `partialize` записва само данните, не функциите.
- В етап 4 при всяка промяна тук ще пренасрочваме нотификациите.

src/store/alertPrefs.ts

48 реда

```

import AsyncStorage from '@react-native-async-storage/async-storage';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

import { PRAYERS, type PrayerId } from '@domain/prayers';

/** Трите състояния на камбанката: сиво / бяло / злато. */
export type AlertMode = 'off' | 'notify' | 'adhan';

const DEFAULT_MODES: Record<PrayerId, AlertMode> = {
  fajr: 'adhan',
  sunrise: 'notify',
  dhuhr: 'notify',
  asr: 'adhan',
  maghrib: 'adhan',
  isha: 'notify',
};

/** off → notify → adhan → off. Изгрев прескача „езан“. */
export function nextMode(id: PrayerId, mode: AlertMode): AlertMode {
  if (mode === 'off') return 'notify';
  if (mode === 'notify') return PRAYERS[id].canAlarm ? 'adhan' : 'off';
  return 'off';
}

interface AlertPrefsState {
  modes: Record<PrayerId, AlertMode>;
  /** Сменя режима и връща новия (за toast-a). */
  cycle: (id: PrayerId) => AlertMode;
}

```

```
export const useAlertPrefs = create<AlertPrefsState>()(
  persist(
    (set, get) => ({
      modes: DEFAULT_MODES,
      cycle: (id) => {
        const mode = nextMode(id, get().modes[id]);
        set((s) => ({ modes: { ...s.modes, [id]: mode } }));
        return mode;
      },
    }),
    {
      name: 'ezan.alert-prefs',
      storage: createJSONStorage(() => AsyncStorage),
      partialize: (s) => ({ modes: s.modes }),
    },
  ),
);
```

useNow.ts src/hooks

- Обновява времето точно на границата на секундата (`1000 - Date.now() % 1000`), а не на произволни интервали. Така всички цифри се сменят едновременно.
- `setTimeout` вместо `setInterval` : всеки път изчислява наново колко остава до следващата секунда и не натрупва закъснение.
- Когато приложението се върне от фона, часът се обновява веднага, без да чака следващия тик.

src/hooks/useNow.ts

32 реда

```
import { useEffect, useState } from 'react';
import { AppState } from 'react-native';

/**
 * Текущото време, обновявано точно на границата на всяка секунда.
 * Когато приложението се върне от фона, часът се обновява веднага.
 */
export function useNow(intervalMs = 1000): Date {
  const [now, setNow] = useState(() => new Date());

  useEffect(() => {
    let timer: ReturnType<typeof setTimeout>;
    const schedule = () => {
      timer = setTimeout(() => {
        setNow(new Date());
        schedule();
      }, intervalMs - (Date.now() % intervalMs));
    };
    schedule();

    const sub = AppState.addListener('change', (state) => {
      if (state === 'active') setNow(new Date());
    });

    return () => {
      clearTimeout(timer);
      sub.remove();
    };
  }, [intervalMs]);
```

```
    return now;
  }
```

usePrayerSchedule.ts src/hooks

- Тежкото изчисление с `adhan` се прави само при смяна на деня (`dayKey`), а не всяка секунда.
- `getSchedule` е евтин и се вика при всеки тик.

src/hooks/usePrayerSchedule.ts

25 реда

```
import { useMemo } from 'react';

import { DEFAULT_LOCATION, DEFAULT_MADHAB, DEFAULT_METHOD } from '@config/defaults';
import { computeThreeDays, getSchedule, type CalcOptions, type Schedule } from '@domain/prayers';

const OPTIONS: CalcOptions = {
  location: DEFAULT_LOCATION,
  method: DEFAULT_METHOD,
  madhab: DEFAULT_MADHAB,
};

/**
 * Часовете се преизчисляват само когато се смени денят,
 * а текущата/следващата молитва – на всяка секунда (евтино е).
 */
export function usePrayerSchedule(now: Date): Schedule {
  const dayKey = `${now.getFullYear()}-${now.getMonth()}-${now.getDate()}`;

  // eslint-disable-next-line react-hooks/exhaustive-deps
  const days = useMemo(() => computeThreeDays(now, OPTIONS), [dayKey]);

  return getSchedule(now, days);
}

export { OPTIONS as CALC_OPTIONS };
```

Стъпка 8

Езици: български и английски

Езикът се избира автоматично от езика на телефона: ако е български, приложението е на български, ако е всеки друг език – на английски. Няма отделна настройка в приложението.

strings.ts src/i18n

- Всички текстове на интерфейса на едно място: имена на молитвите, табове, дати, месеци по Хиджра, текстове за TalkBack/VoiceOver.
- `en` е типизиран като `Strings` (формата на `bg`). Ако добавиш текст само на единия език, TypeScript дава грешка още при писането.
- Текстовете с променливи са функции (`at: (time) => `в ${time}``). Така словоредът може да е различен за всеки език.

- Арабските имена (الفجر ...) не се превеждат – те са в PRAYERS и са еднакви на двата езика.

src/i18n/strings.ts

142 реда

```
import type { PrayerId } from '@domain/prayers';
import type { AlertMode } from '@store/alertPrefs';

/**
 * Всички текстове на приложението на двата езика.
 * `en` е типизиран като `Strings` (формата на `bg`), затова TypeScript
 * дава грешка, ако някой ключ липсва в единия от езиците.
 */
export const bg = {
  prayers: {
    fajr: 'Фаджр',
    sunrise: 'Изгрев',
    dhuhr: 'Зухр',
    asr: 'Асп',
    maghrib: 'Магриб',
    isha: 'Иша',
  } as Record<PrayerId, string>,

  alert: {
    off: 'без известие',
    notify: 'нотификация',
    adhan: 'езан',
  } as Record<AlertMode, string>,

  tabs: {
    today: 'Днес',
    qibla: 'Кибла',
    month: 'Месец',
    settings: 'Настройки',
  },

  hero: {
    next: 'Следваща',
    at: (time: string) => `в ${time}`,
    tomorrowAt: (time: string) => `утре в ${time}`,
    remaining: (time: string) => `Остават ${time}`,
  },

  nowPill: 'СЕГА',

  ally: {
    city: (city: string) => `Град: ${city}. Смени града`,
    bell: (prayer: string, mode: string) => `${prayer}: ${mode}. Докосни за смяна`,
  },

  toast: (prayer: string, mode: string) => `${prayer}: ${mode}`,

  placeholder: {
    qibla: 'Компас към Кябе. Идва в етап 7 от плана.',
    month: 'Таблица с часовете за целия месец. Идва в етап 2 от плана.',
    settings: 'Метод, Асп, звуци, напомняне. Идва в етап 3 от плана.',
  },

  date: {
    weekdaysShort: ['Нд', 'Пн', 'Вт', 'Ср', 'Чт', 'Пт', 'Сб'],
    // съкращенията са с точка; „март“, „май“, „юни“, „юли“ са пълни думи
    monthsShort: ['яну.', 'фев.', 'март', 'апр.', 'май', 'юни', 'юли', 'авг.', 'септ.', 'окт.', 'ное.'],
  },
}
```

```

    'дек.'],
  },

  hijriMonths: [
    'Мухаррем',
    'Сафер',
    'Ребиул-евел',
    'Ребиул-ахир',
    'Джемазиел-евел',
    'Джемазиел-ахир',
    'Реджеб',
    'Шабан',
    'Рамазан',
    'Шеввал',
    'Зилкаде',
    'Зилхидже',
  ],
};

export type Strings = typeof bg;

export const en: Strings = {
  prayers: {
    fajr: 'Fajr',
    sunrise: 'Sunrise',
    dhuhr: 'Dhuhr',
    asr: 'Asr',
    maghrib: 'Maghrib',
    isha: 'Isha',
  },

  alert: {
    off: 'no alert',
    notify: 'notification',
    adhan: 'adhan',
  },

  tabs: {
    today: 'Today',
    qibla: 'Qibla',
    month: 'Month',
    settings: 'Settings',
  },

  hero: {
    next: 'Next',
    at: (time) => `at ${time}`,
    tomorrowAt: (time) => `tomorrow at ${time}`,
    remaining: (time) => `${time} left`,
  },

  nowPill: 'NOW',

  ally: {
    city: (city) => `City: ${city}. Change city`,
    bell: (prayer, mode) => `${prayer}: ${mode}. Tap to change`,
  },

  toast: (prayer, mode) => `${prayer}: ${mode}`,

  placeholder: {

```

```

    qibla: 'Compass pointing to the Kaaba. Coming in stage 7.',
    month: 'Prayer times for the whole month. Coming in stage 2.',
    settings: 'Method, Asr, sounds, reminders. Coming in stage 3.',
  },

  date: {
    weekdaysShort: ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'],
    monthsShort: ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'],
  },

  hijriMonths: [
    'Muharram',
    'Safar',
    'Rabi al-Awwal',
    'Rabi al-Thani',
    'Jumada al-Ula',
    'Jumada al-Akhirah',
    'Rajab',
    'Shaban',
    'Ramadan',
    'Shawwal',
    'Dhu al-Qadah',
    'Dhu al-Hijjah',
  ],
};

```

index.ts src/i18n

- `resolveLang` е цялото правило: `bg` → български, всичко останало → английски.
- `useLocales()` от `expo-localization` връща езиците на телефона по реда на предпочитание. Гледаме само първия, защото той е езикът на системата.
- Когато потребителят смени езика, `hook`-ът прерисува екрана веднага. На Android приложението не се рестартира (Ехро добавя `locale` в `configChanges`). На iOS системата затваря приложението и при следващо отваряне то е на новия език.
- Ползва се във всеки компонент с текст: `const { t } = useI18n();` и после `t.prayers.fajr`, `t.hero.at('13:22')`.

src/i18n/index.ts

28 реда

```

import { useLocales } from 'expo-localization';

import { bg, en, type Strings } from './strings';

export type Lang = 'bg' | 'en';

const STRINGS: Record<Lang, Strings> = { bg, en };

/**
 * Правилото: ако основният език на телефона е български – български,
 * за всеки друг език – английски.
 */
export function resolveLang(languageCode: string | null | undefined): Lang {
  return languageCode?.toLowerCase() === 'bg' ? 'bg' : 'en';
}

/**
 * Текущият език и текстовете за него.
 * `useLocales()` прерисува компонента, когато потребителят смени езика на телефона

```

```

* (или езика само за това приложение - Android 13+ / iOS).
*/
export function useI18n(): { lang: Lang; t: Strings } {
  const locales = useLocales();
  const lang = resolveLang(locales[0]?.languageCode);
  return { lang, t: STRINGS[lang] };
}

export type { Strings };

```

Градовете не са в `strings.ts`, защото са данни, а не текстове на интерфейса. Всеки град носи името си на двата езика (`names: { bg: 'София', en: 'Sofia' }` в `src/config/default.ts`). Така в етап 3 списъкът с градове ще е един файл.

Стъпка 9

Графика: фон, шарка, пръстен

Визуалните елементи от mockup-а, пренесени от HTML/CSS към react-native-svg.

icons.tsx src/components

- Същите SVG икони като в mockup-а. `AlertIcon` рисува трите състояния на камбанката.
- Собствени SVG икони вместо библиотека с икони – визията съвпада точно с дизайна и не добавяме зависимост.

src/components/icons.tsx

99 реда

```

import Svg, { Circle, Path, Rect } from 'react-native-svg';

import type { AlertMode } from '@store/alertPrefs';

interface IconProps {
  size?: number;
  color: string;
  strokeWidth?: number;
}

const stroke = (color: string, strokeWidth = 1.8) => ({
  fill: 'none',
  stroke: color,
  strokeWidth,
  strokeLinecap: 'round' as const,
  strokeLinejoin: 'round' as const,
});

/** Камбанка за трите режима: зачеркната / камбанка / високоговорител. */
export function AlertIcon({ mode, size = 19, color }: IconProps & { mode: AlertMode }) {
  const s = stroke(color);
  if (mode === 'adhan') {
    return (
      <Svg width={size} height={size} viewBox="0 0 24 24">
        <Path d="M4 10v4h4l5 4v6L8 10z" {...s} />
        <Path d="M16.5 8.5a5 5 0 0 1 0 7" {...s} />
        <Path d="M19 6a8.5 8.5 0 0 1 0 12" {...s} />
      </Svg>
    );
  }
  return (

```

```

    <Svg width={size} height={size} viewBox="0 0 24 24">
      <Path d="M6 16v-5a6 6 0 0 1 12 0v5l1.5 2h-15z" {...s} />
      <Path d="M10 20.5a2 2 0 0 0 4 0" {...s} />
      {mode === 'off' && <Path d="M4 4l16 16" {...s} />}
    </Svg>
  );
}

export function PinIcon({ size = 15, color }: IconProps) {
  const s = stroke(color, 2);
  return (
    <Svg width={size} height={size} viewBox="0 0 24 24">
      <Path d="M12 21s-7-6.2-7-11.5a7 7 0 0 1 14 0C19 14.8 12 21 12 21z" {...s} />
      <Circle cx="12" cy="9.5" r="2.5" {...s} />
    </Svg>
  );
}

export function ChevronDownIcon({ size = 12, color }: IconProps) {
  return (
    <Svg width={size} height={size} viewBox="0 0 24 24">
      <Path d="M6 9l6 6 6-6" {...stroke(color, 2.4)} />
    </Svg>
  );
}

/* ---- иконите на долната навигация ---- */

export function TodayIcon({ size = 22, color }: IconProps) {
  const s = stroke(color, 1.7);
  return (
    <Svg width={size} height={size} viewBox="0 0 24 24">
      <Circle cx="12" cy="12" r="8.5" {...s} />
      <Path d="M12 7.5V12l3 2" {...s} />
    </Svg>
  );
}

export function QiblaIcon({ size = 22, color }: IconProps) {
  const s = stroke(color, 1.7);
  return (
    <Svg width={size} height={size} viewBox="0 0 24 24">
      <Circle cx="12" cy="12" r="8.5" {...s} />
      <Path d="M14.8 9.2l-1.6 4-4 1.6 1.6-4z" {...s} />
    </Svg>
  );
}

export function MonthIcon({ size = 22, color }: IconProps) {
  const s = stroke(color, 1.7);
  return (
    <Svg width={size} height={size} viewBox="0 0 24 24">
      <Rect x="4" y="5.5" width="16" height="14" rx="2.5" {...s} />
      <Path d="M4 10h16M8.5 3.5v4M15.5 3.5v4" {...s} />
    </Svg>
  );
}

export function SettingsIcon({ size = 22, color }: IconProps) {
  const s = stroke(color, 1.7);
  return (

```

```

<Svg width={size} height={size} viewBox="0 0 24 24">
  <Path d="M4 7h9M17 7h3M4 17h3M11 17h9" {...s} />
  <Circle cx="15" cy="7" r="2" {...s} />
  <Circle cx="9" cy="17" r="2" {...s} />
</Svg>
);
}

```

GeometricPattern.tsx src/components

- SVG `Pattern` повтаря плочка 60×60 с 8-лъчева звезда по целия екран. Рисува се един път и не зарежда картинка.
- Прозрачността 7.5% е в един параметър, за да се настройва лесно.

src/components/GeometricPattern.tsx

26 реда

```

import { StyleSheet } from 'react-native';
import Svg, { Circle, Defs, G, Path, Pattern, Rect } from 'react-native-svg';

import { colors } from '@theme/colors';

/**
 * Геометрична шарка (8-лъчева звезда от два квадрата), повторена като плочки 60×60.
 * Рисува се в злато с ~7.5% прозрачност върху градиента.
 */
export function GeometricPattern({ opacity = 0.075 }: { opacity?: number }) {
  return (
    <Svg width="100%" height="100%" style={StyleSheet.absoluteFill} pointerEvents="none">
      <Defs>
        <Pattern id="girih" patternUnits="userSpaceOnUse" width={60} height={60}>
          <G fill="none" stroke={colors.gold} strokeWidth={1}>
            <Path d="M16 16h28v28H16z" />
            <Path d="M30 10.2L49.8 30L30 49.8L10.2 30z" />
            <Path d="M0 0L16 16M60 0L44 16M0 60L16 44M60 60L44 44M30 0V10.2M30 49.8V60M0 30H10.2M49.8 30H60" />
            <Circle cx={30} cy={30} r={6} />
          </G>
        </Pattern>
      </Defs>
      <Rect x={0} y={0} width="100%" height="100%" fill="url(#girih)" opacity={opacity} />
    </Svg>
  );
}

```

PhaseBackground.tsx src/components

- Два слоя градиент: долу е предишната молитва, горе – текущата. При смяна горният слой се появява от 0 до 1 за 1.1 секунди.
- Смяната на state по време на render (`if (layers.above !== phase) setLayers(...)`) е официалният React модел за „реагирай на промяна на prop“. Не прави излишен render както `useEffect`.
- Анимацията е с `Reanimated` (`useSharedValue`, `withTiming`) и върви на UI нишката.

src/components/PhaseBackground.tsx

50 реда

```

import { LinearGradient } from 'expo-linear-gradient';
import { useEffect, useState } from 'react';
import { StyleSheet, View } from 'react-native';

```

```

import Animated, { useAnimatedStyle, useSharedValue, withTiming } from 'react-native-reanimated';

import type { PrayerId } from '@domain/prayers';
import { GRADIENT_LOCATIONS, PHASE_GRADIENTS } from '@theme/gradients';

import { GeometricPattern } from './GeometricPattern';

function Gradient({ phase }: { phase: PrayerId }) {
  return (
    <LinearGradient
      colors={PHASE_GRADIENTS[phase]}
      locations={GRADIENT_LOCATIONS}
      style={StyleSheet.absoluteFill}
    />
  );
}

/**
 * Градиентът на текущата молитва на целия екран (решение №3).
 * При смяна на молитвата новият градиент плавно се появява върху стария (1.1 s).
 */
export function PhaseBackground({ phase }: { phase: PrayerId }) {
  // „below“ е предишната молитва, „above“ – текущата, която се появява отгоре.
  const [layers, setLayers] = useState({ below: phase, above: phase });
  if (layers.above !== phase) {
    // Обновяване на state по време на render при смяна на prop – официално разрешен React модел.
    setLayers({ below: layers.above, above: phase });
  }

  const opacity = useSharedValue(1);
  useEffect(() => {
    opacity.set(0);
    opacity.set(withTiming(1, { duration: 1100 }));
  }, [layers.above, opacity]);

  const aboveStyle = useAnimatedStyle(() => ({ opacity: opacity.get() }));

  return (
    <View style={StyleSheet.absoluteFill} pointerEvents="none">
      <Gradient phase={layers.below} />
      <Animated.View style={[StyleSheet.absoluteFill, aboveStyle]}>
        <Gradient phase={layers.above} />
      </Animated.View>
      <GeometricPattern />
    </View>
  );
}

```

Skyline.tsx src/components

- Силуетът с купола и двете минарета, 22% черно.

src/components/Skyline.tsx

25 реда

```

import { StyleSheet } from 'react-native';
import Svg, { Path } from 'react-native-svg';

import { colors } from '@theme/colors';

/** Силует на купол с две минарета зад таймера. */

```

```
export function Skyline() {
  return (
    <Svg
      style={styles.svg}
      viewBox="0 0 360 80"
      preserveAspectRatio="none"
      pointerEvents="none"
    >
      <Path
        fill={colors.skyline}
        d="M0 80V70H56V24L60 8L64 24V70H120V58C120 36 148 22 179 18V8H181V18C212 22 240 36 240
58V70H292V24L296 8L300 24V70H360V80Z"
      />
    </Svg>
  );
}

const styles = StyleSheet.create({
  svg: { position: 'absolute', left: 0, right: 0, bottom: 0, width: '100%', height: 80 },
});
```

CountdownRing.tsx src/components

- Два кръга: сива пътека и златен прогрес. Прогресът се рисува със `strokeDasharray` / `strokeDashoffset` – колкото по-голям offset, толкова по-къса видимата дъга.
- SVG-то е завъртяно на -90° , за да тръгва от 12 часа. Съдържанието в средата е отделен `View` отгоре, за да не се върти текстът.

src/components/CountdownRing.tsx

59 реда

```
import type { ReactNode } from 'react';
import { StyleSheet, View } from 'react-native';
import Svg, { Circle } from 'react-native-svg';

import { colors } from '@theme/colors';

interface Props {
  progress: number; // 0..1
  size?: number;
  strokeWidth?: number;
  children?: ReactNode;
}

/** Кръгов прогрес: колко от времето до следващата молитва е изминало. */
export function CountdownRing({ progress, size = 214, strokeWidth = 5, children }: Props) {
  const r = (size - strokeWidth) / 2 - 2;
  const c = 2 * Math.PI * r;
  const offset = c * (1 - Math.min(1, Math.max(0, progress)));

  return (
    <View style={{ width: size, height: size }}>
      {/* -90°, за да започва от 12 часа */}
      <Svg width={size} height={size} style={{ transform: [{ rotate: '-90deg' }] }}>
        <Circle
          cx={size / 2}
          cy={size / 2}
          r={r}
          fill="none"
          stroke={colors.ringTrack}
        />
      </Svg>
      {children}
    </View>
  );
}
```

```

        strokeWidth={strokeWidth}
      />
    <Circle
      cx={size / 2}
      cy={size / 2}
      r={r}
      fill="none"
      stroke={colors.gold}
      strokeWidth={strokeWidth}
      strokeLinecap="round"
      strokeDasharray={` ${c} ${c}`}
      strokeDashoffset={offset}
    />
  </Svg>
  <View style={styles.center}>{children}</View>
</View>
);
}

const styles = StyleSheet.create({
  center: {
    position: 'absolute',
    top: 0,
    right: 0,
    bottom: 0,
    left: 0,
    alignItems: 'center',
    justifyContent: 'center',
  },
});

```

Стъпка 10

Компоненти на екрана „Днес“

Отгоре надолу, в реда, в който ги виждаш на екрана.

TopBar.tsx src/components

- Бутонът с града (в етап 3 ще отваря избора на град) и двете дати вдясно.
- `accessibilityLabel` – TalkBack/VoiceOver чете „Град: София. Смени града“.

src/components/TopBar.tsx

63 реда

```

import { Pressable, StyleSheet, Text, View } from 'react-native';

import { useI18n } from '@/i18n';
import { colors } from '@/theme/colors';
import { fonts } from '@/theme/typography';

import { ChevronDownIcon, PinIcon } from './icons';

interface Props {
  city: string;
  gregorian: string;
  hijri: string;
  onCityPress?: () => void;
}

```

```

export function TopBar({ city, gregorian, hijri, onCityPress }: Props) {
  const { t } = useI18n();
  return (
    <View style={styles.bar}>
      <Pressable
        onPress={onCityPress}
        style={({ pressed }) => [styles.city, pressed && styles.pressed]}
        accessibilityRole="button"
        accessibilityLabel={t.a11y.city(city)}
      >
        <PinIcon color={colors.text} />
        <Text style={styles.cityText}>{city}</Text>
        <ChevronDownIcon color={colors.text} />
      </Pressable>
      <View style={styles.dates}>
        <Text style={styles.gregorian}>{gregorian}</Text>
        <Text style={styles.hijri}>{hijri}</Text>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  bar: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingHorizontal: 18,
    paddingTop: 8,
    paddingBottom: 4,
    gap: 8,
  },
  city: {
    flexDirection: 'row',
    alignItems: 'center',
    gap: 6,
    paddingVertical: 7,
    paddingLeft: 10,
    paddingRight: 12,
    borderRadius: 999,
    backgroundColor: colors.pill,
  },
  pressed: { opacity: 0.7 },
  cityText: { fontFamily: fonts.bold, fontSize: 14, color: colors.text },
  dates: { alignItems: 'flex-end', flexShrink: 1 },
  gregorian: { fontFamily: fonts.bold, fontSize: 12, lineHeight: 16, color: colors.text },
  hijri: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 16, color: colors.textDim },
});

```

NextPrayerHero.tsx src/components

- Съединява силуета, пръстена и текста в средата: арабско име, „Следваща · Магриб“, обратно броене и часа.
- След Иша часът е с префикс „утре в“.

src/components/NextPrayerHero.tsx

71 реда

```

import { StyleSheet, Text, View } from 'react-native';

import { formatCountdown, formatHM } from '@domain/format';

```

```

import { PRAYERS, type Schedule } from '@domain/prayers';
import { useI18n } from '@i18n';
import { colors } from '@theme/colors';
import { fonts, tabularNums } from '@theme/typography';

import { CountdownRing } from './CountdownRing';
import { Skyline } from './Skyline';

export function NextPrayerHero({ schedule }: { schedule: Schedule }) {
  const { t } = useI18n();
  const meta = PRAYERS[schedule.next.id];
  const time = formatHM(schedule.next.time);
  const at = schedule.isNextTomorrow ? t.hero.tomorrowAt(time) : t.hero.at(time);
  const countdown = formatCountdown(schedule.remainingMs);

  return (
    <View style={styles.hero}>
      <Skyline />
      <CountdownRing progress={schedule.progress}>
        <Text style={styles.arabic}>{meta.arabic}</Text>
        <Text style={styles.label}>
          {t.hero.next} · <Text style={styles.labelStrong}>{t.prayers[schedule.next.id]}</Text>
        </Text>
        <Text style={styles.count} accessibilityLabel={t.hero.remaining(countdown)}>
          {countdown}
        </Text>
        <Text style={styles.at}>{at}</Text>
      </CountdownRing>
    </View>
  );
}

const styles = StyleSheet.create({
  hero: {
    height: 252,
    alignItems: 'center',
    justifyContent: 'center',
  },
  arabic: {
    fontFamily: fonts.arabic,
    fontSize: 26,
    lineHeight: 36,
    color: colors.gold,
  },
  label: {
    fontFamily: fonts.semibold,
    fontSize: 12.5,
    color: 'rgba(242,239,232,0.78)',
  },
  labelStrong: {
    fontFamily: fonts.extrabold,
    color: colors.text,
  },
  count: {
    fontFamily: fonts.extrabold,
    fontSize: 38,
    lineHeight: 46,
    letterSpacing: -0.6,
    color: colors.text,
    ...tabularNums,
  },
});

```

```

at: {
  fontFamily: fonts.medium,
  fontSize: 12.5,
  color: colors.textDim,
  ...tabularNums,
},
});

```

BellButton.tsx src/components

- Един бутон, три стила. `styles[mode]` избира фона директно по режима.
- `hitSlop` уголемява зоната за докосване – кръгът е 36 px, а Android препоръчва поне 48.

src/components/BellButton.tsx

44 реда

```

import { Pressable, StyleSheet } from 'react-native';

import type { AlertMode } from '@store/alertPrefs';
import { colors } from '@theme/colors';

import { AlertIcon } from './icons';

interface Props {
  mode: AlertMode;
  /** Готов текст за TalkBack/VoiceOver на текущия език. */
  accessibilityLabel: string;
  onPress: () => void;
}

/** Сиво = изключено, бяло = нотификация, злато = езан. */
export function BellButton({ mode, accessibilityLabel, onPress }: Props) {
  const iconColor = mode === 'adhan' ? colors.goldInk : mode === 'notify' ? colors.text : colors.muted;

  return (
    <Pressable
      onPress={onPress}
      hitSlop={6}
      style={({ pressed }) => [styles.btn, styles[mode], pressed && styles.pressed]}
      accessibilityRole="button"
      accessibilityLabel={accessibilityLabel}
    >
      <AlertIcon mode={mode} color={iconColor} />
    </Pressable>
  );
}

const styles = StyleSheet.create({
  btn: {
    width: 36,
    height: 36,
    borderRadius: 18,
    alignItems: 'center',
    justifyContent: 'center',
  },
  off: { backgroundColor: 'transparent' },
  notify: { backgroundColor: colors.bellNotify },
  adhan: { backgroundColor: colors.gold },
  pressed: { transform: [{ scale: 0.92 }] },
});

```

PrayerRow.tsx src/components

- Ред от списъка: име, арабско име, пил „СЕГА“, час и камбанка.
- Минали молитви са с 50% прозрачност, текущата е със златна рамка и златен час – както в mockup-а.

src/components/PrayerRow.tsx

85 реда

```
import { StyleSheet, Text, View } from 'react-native';

import { formatHM } from '@domain/format';
import { PRAYERS, type PrayerTime, type RowState } from '@domain/prayers';
import { useI18n } from '@i18n';
import type { AlertMode } from '@store/alertPrefs';
import { colors } from '@theme/colors';
import { fonts, tabularNums } from '@theme/typography';

import { BellButton } from './BellButton';

interface Props {
  prayer: PrayerTime;
  state: RowState;
  mode: AlertMode;
  onBellPress: () => void;
}

export function PrayerRow({ prayer, state, mode, onBellPress }: Props) {
  const { t } = useI18n();
  const meta = PRAYERS[prayer.id];
  const name = t.prayers[prayer.id];
  const isNow = state === 'now';

  return (
    <View style={[styles.row, isNow && styles.now, state === 'past' && styles.past]}>
      <View style={styles.names}>
        <Text style={[styles.name, isNow && styles.nameNow]}>{name}</Text>
        <Text style={styles.arabic}>{meta.arabic}</Text>
        {isNow && (
          <View style={styles.pill}>
            <Text style={styles.pillText}>{t.nowPill}</Text>
          </View>
        )}
      </View>
      <Text style={[styles.time, isNow && styles.timeNow]}>{formatHM(prayer.time)}</Text>
      <BellButton
        mode={mode}
        accessibilityLabel={t.a11y.bell(name, t.alert[mode])}
        onPress={onBellPress}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  row: {
    flexDirection: 'row',
    alignItems: 'center',
    height: 46,
    paddingLeft: 14,
    paddingRight: 6,
    borderRadius: 15,
    gap: 12,
```

```

},
now: {
  backgroundColor: colors.nowRow,
  borderWidth: 1,
  borderColor: colors.nowRowBorder,
},
past: { opacity: 0.5 },
names: { flex: 1, flexDirection: 'row', alignItems: 'center', gap: 8, minWidth: 0 },
name: { fontFamily: fonts.semibold, fontSize: 15, color: colors.text },
nameNow: { fontFamily: fonts.extrabold },
arabic: {
  fontFamily: fonts.arabic,
  fontSize: 15,
  lineHeight: 22,
  color: 'rgba(242,239,232,0.5)',
},
pill: {
  backgroundColor: colors.gold,
  borderRadius: 999,
  paddingHorizontal: 7,
  paddingVertical: 2,
},
pillText: {
  fontFamily: fonts.extrabold,
  fontSize: 10,
  letterSpacing: 0.8,
  color: colors.goldInk,
},
time: { fontFamily: fonts.bold, fontSize: 16, color: colors.text, ...tabularNums },
timeNow: { color: colors.gold },
});

```

PrayerList.tsx src/components

- Полупрозрачната карта с шестте реда. Чете режимите директно от store-a, затова промяна на камбанка прерисува само списъка.

src/components/PrayerList.tsx

42 реда

```

import { StyleSheet, View } from 'react-native';

import type { PrayerId, Schedule } from '@domain/prayers';
import { useAlertPrefs } from '@store/alertPrefs';
import { colors } from '@theme/colors';

import { PrayerRow } from './PrayerRow';

interface Props {
  schedule: Schedule;
  onBellPress: (id: PrayerId) => void;
}

export function PrayerList({ schedule, onBellPress }: Props) {
  const modes = useAlertPrefs((s) => s.modes);

  return (
    <View style={styles.card}>
      {schedule.today.map((p) => (
        <PrayerRow
          key={p.id}

```

```

        prayer={p}
        state={schedule.rowState(p.id)}
        mode={modes[p.id]}
        onBellPress={() => onBellPress(p.id)}
      />
    ))}
  </View>
);
}

const styles = StyleSheet.create({
  card: {
    marginHorizontal: 14,
    padding: 6,
    gap: 2,
    borderRadius: 22,
    backgroundColor: colors.card,
    borderWidth: 1,
    borderColor: colors.cardBorder,
  },
});

```

Toast.tsx src/components

- „Магриб: езан“ за 1.2 секунди след тап на камбанка.
- Всяко съобщение има ново `id`, за да се покаже отново дори при същия текст.

src/components/Toast.tsx

65 реда

```

import { useEffect } from 'react';
import { StyleSheet, Text } from 'react-native';
import Animated, {
  useAnimatedStyle,
  useSharedValue,
  withDelay,
  withSequence,
  withTiming,
} from 'react-native-reanimated';

import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

interface Props {
  /** Ново съобщение идва с нов `id`, за да се покаже отново и при същия текст. */
  message: { id: number; text: string } | null;
  bottom: number;
}

/** Кратко съобщение над долната навигация: появява се, стои 1.2 s и изчезва. */
export function Toast({ message, bottom }: Props) {
  const progress = useSharedValue(0);

  useEffect(() => {
    if (!message) return;
    progress.set(0);
    progress.set(
      withSequence(
        withTiming(1, { duration: 180 }),
        withDelay(1200, withTiming(0, { duration: 220 })),
      ),
    ),
  },

```

```

    );
    }, [message, progress]);

    const animatedStyle = useAnimatedStyle(() => ({
      opacity: progress.get(),
      transform: [{ translateY: (1 - progress.get()) * 8 }],
    }));

    if (!message) return null;

    return (
      <Animated.View
        pointerEvents="none"
        accessibilityLiveRegion="polite"
        style={[styles.toast, { bottom }, animatedStyle]}
      >
        <Text style={styles.text}>{message.text}</Text>
      </Animated.View>
    );
  }

  const styles = StyleSheet.create({
    toast: {
      position: 'absolute',
      alignSelf: 'center',
      paddingHorizontal: 14,
      paddingVertical: 8,
      borderRadius: 999,
      backgroundColor: 'rgba(8,12,22,0.92)',
      borderWidth: 1,
      borderColor: 'rgba(255,255,255,0.12)',
    },
    text: { fontFamily: fonts.semibold, fontSize: 12.5, color: colors.text },
  });

```

TabBar.tsx src/components

- Собствена долна навигация вместо стандартната, за да съвпада с дизайна.
- `position: 'absolute'` позволява на градиента да минава под лентата. Затова екраните добавят отстъп отдолу чрез `useTabBarHeight()`.
- `navigation.emit('tabPress')` запазва стандартното поведение на табовете (например връщане в началото при повторен тап).

src/components/TabBar.tsx

80 реда

```

import type { BottomTabBarProps } from 'expo-router/js-tabs';
import type { ComponentType } from 'react';
import { Pressable, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { colors } from '@/theme/colors';
import { fonts } from '@/theme/typography';

import { MonthIcon, QiblaIcon, SettingsIcon, TodayIcon } from './icons';

const ICONS: Record<string, ComponentType<{ color: string }>> = {
  index: TodayIcon,
  qibla: QiblaIcon,
  month: MonthIcon,

```

```

    settings: SettingsIcon,
  };

  const BAR_HEIGHT = 58;

  /** Височината на лентата + долния безопасен отстъп. Екраните я ползват за padding отдолу. */
  export function useTabBarHeight() {
    const insets = useSafeAreaInsets();
    return BAR_HEIGHT + Math.max(insets.bottom, 12);
  }

  /**
   * Собствена долна навигация. Позиционирана е `absolute`, за да минава
   * градиентът на екрана под нея, както в тоскуп-а.
   */
  export function TabBar({ state, descriptors, navigation }: BottomTabBarProps) {
    const insets = useSafeAreaInsets();

    return (
      <View style={[styles.bar, { paddingBottom: Math.max(insets.bottom, 12) }]}>
        {state.routes.map((route, index) => {
          const focused = state.index === index;
          const { options } = descriptors[route.key];
          const label = options.title ?? route.name;
          const Icon = ICONS[route.name] ?? TodayIcon;
          const color = focused ? colors.gold : colors.muted;

          const onPress = () => {
            const event = navigation.emit({ type: 'tabPress', target: route.key, canPreventDefault: true });
            if (!focused && !event.defaultPrevented) navigation.navigate(route.name, route.params);
          };

          return (
            <Pressable
              key={route.key}
              onPress={onPress}
              style={styles.item}
              accessibilityRole="tab"
              accessibilityState={{ selected: focused }}
              accessibilityLabel={label}
            >
              <Icon color={color} />
              <Text style={[styles.label, { color }]}>{label}</Text>
            </Pressable>
          );
        })}
      </View>
    );
  }

  const styles = StyleSheet.create({
    bar: {
      position: 'absolute',
      left: 0,
      right: 0,
      bottom: 0,
      flexDirection: 'row',
      paddingTop: 8,
      paddingHorizontal: 8,
      backgroundColor: colors.tabBar,
      borderTopWidth: StyleSheet.hairlineWidth,

```

```
borderTopColor: 'rgba(255,255,255,0.10)',
},
item: { flex: 1, alignItems: 'center', gap: 4, paddingVertical: 4 },
label: { fontFamily: fonts.semibold, fontSize: 10.5 },
});
```

PlaceholderScreen.tsx src/components

- Временен екран за табовете, които идват в следващите етапи.

src/components/PlaceholderScreen.tsx

25 реда

```
import { StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

import { GeometricPattern } from './GeometricPattern';

/** Временен екран за табовете, които идват в следващите етапи. */
export function PlaceholderScreen({ title, note }: { title: string; note: string }) {
  const insets = useSafeAreaInsets();
  return (
    <View style={[styles.root, { paddingTop: insets.top + 12 }]}>
      <GeometricPattern />
      <Text style={styles.title}>{title}</Text>
      <Text style={styles.note}>{note}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base, paddingHorizontal: 20, gap: 8 },
  title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text },
  note: { fontFamily: fonts.regular, fontSize: 14, lineHeight: 21, color: colors.muted },
});
```

Стъпка 11

Навигация и екрани

Накрая сглобяваме всичко с Expo Router.

_layout.tsx src/app

- Коренът на приложението. `SplashScreen.preventAutoHideAsync()` държи splash екрана, докато шрифтовете се заредят, за да не мигне текст със системния шрифт.
- При грешка в шрифтовете приложението пак тръгва (`loaded || error`), само че със системния шрифт.
- `StatusBar style="light"` – светли икони горе, защото фонът е винаги тъмен.

src/app/_layout.tsx

48 реда

```
import { Amiri_400Regular, Amiri_700Bold } from '@expo-google-fonts/amiri';
import {
  Manrope_400Regular,
```

```

    Manrope_500Medium,
    Manrope_600SemiBold,
    Manrope_700Bold,
    Manrope_800ExtraBold,
  } from '@expo-google-fonts/manrope';
import { useFonts } from 'expo-font';
import { Stack } from 'expo-router';
import * as SplashScreen from 'expo-splash-screen';
import { StatusBar } from 'expo-status-bar';
import { useEffect } from 'react';

import { colors } from '@theme/colors';

// Splash екранът стои, докато шрифтовете се заредят.
SplashScreen.preventAutoHideAsync();

export default function RootLayout() {
  const [loaded, error] = useFonts({
    Manrope_400Regular,
    Manrope_500Medium,
    Manrope_600SemiBold,
    Manrope_700Bold,
    Manrope_800ExtraBold,
    Amiri_400Regular,
    Amiri_700Bold,
  });

  useEffect(() => {
    if (loaded || error) SplashScreen.hideAsync();
  }, [loaded, error]);

  if (!loaded && !error) return null;

  return (
    <>
      <StatusBar style="light" />
      <Stack
        screenOptions={{
          headerShown: false,
          contentStyle: { backgroundColor: colors.base },
        }}
      />
    </>
  );
}

```

_layout.tsx src/app/(tabs)

- Четирите таба. Ползваме `expo-router/js-tabs` (JavaScript табове), защото позволяват напълно собствена лента. Има и native табове, но те изглеждат като системните.
- Имената на файловете (`index`, `qibla` ...) са имената на маршрутите. По тях `TabBar` избира иконата.

src/app/(tabs)/_layout.tsx

23 реда

```

import { Tabs } from 'expo-router/js-tabs';

import { TabBar } from '@components/TabBar';
import { useI18n } from '@i18n';
import { colors } from '@theme/colors';

```

```
export default function TabsLayout() {
  const { t } = useI18n();
  return (
    <Tabs
      tabBar={props => <TabBar {...props} />}
      screenOptions={{
        headerShown: false,
        sceneStyle: { backgroundColor: colors.base },
      }}
    >
      <Tabs.Screen name="index" options={{ title: t.tabs.today }} />
      <Tabs.Screen name="qibla" options={{ title: t.tabs.qibla }} />
      <Tabs.Screen name="month" options={{ title: t.tabs.month }} />
      <Tabs.Screen name="settings" options={{ title: t.tabs.settings }} />
    </Tabs>
  );
}
```

index.tsx src/app/(tabs)

- Екранът „Днес“. Взима времето (`useNow`), изчислява графика (`usePrayerSchedule`) и го подава на компонентите.
- `PhaseBackground` е извън `ScrollView` – фонът стои, а съдържанието се скролва на малки екрани.
- Отстъпите горе и долу идват от safe area и от височината на лентата с табове, затова нищо не се скрива под status bar-а или под навигацията.

src/app/(tabs)/index.tsx

63 реда

```
import { useCallback, useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { NextPrayerHero } from '@components/NextPrayerHero';
import { PhaseBackground } from '@components/PhaseBackground';
import { PrayerList } from '@components/PrayerList';
import { useTabBarHeight } from '@components/TabBar';
import { Toast } from '@components/Toast';
import { TopBar } from '@components/TopBar';
import { DEFAULT_LOCATION } from '@config/defaults';
import { formatGregorianShort } from '@domain/format';
import { formatHijri } from '@domain/hijri';
import type { PrayerId } from '@domain/prayers';
import { useNow } from '@hooks/useNow';
import { usePrayerSchedule } from '@hooks/usePrayerSchedule';
import { useI18n } from '@i18n';
import { useAlertPrefs } from '@store/alertPrefs';

export default function TodayScreen() {
  const insets = useSafeAreaInsets();
  const { lang, t } = useI18n();
  const tabBarHeight = useTabBarHeight();

  const now = useNow();
  const schedule = usePrayerSchedule(now);

  const cycle = useAlertPrefs((s) => s.cycle);
  const [toast, setToast] = useState<{ id: number; text: string } | null>(null);

  const onBellPress = useCallback(
```

```

(id: PrayerId) => {
  const mode = cycle(id);
  setToast({ id: Date.now(), text: t.toast(t.prayers[id], t.alert[mode]) });
},
[cycle, t],
);

return (
  <View style={styles.root}>
    <PhaseBackground phase={schedule.current.id} />

    <ScrollView
      contentContainerStyle={{ paddingTop: insets.top, paddingBottom: tabBarHeight + 16 }}
      showsVerticalScrollIndicator={false}
    >
      <TopBar
        city={DEFAULT_LOCATION.names[lang]}
        gregorian={formatGregorianShort(now, t.date)}
        hijri={formatHijri(now, t.hijriMonths)}
      />
      <NextPrayerHero schedule={schedule} />
      <PrayerList schedule={schedule} onBellPress={onBellPress} />
    </ScrollView>

    <Toast message={toast} bottom={tabBarHeight + 14} />
  </View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1 },
});

```

qibla.tsx src/app/(tabs)

- Празни екрани засега. Същото е и за `month.tsx` и `settings.tsx`.

src/app/(tabs)/qibla.tsx

7 реда

```

import { PlaceholderScreen } from '@components/PlaceholderScreen';
import { useI18n } from '@i18n';

export default function QiblaScreen() {
  const { t } = useI18n();
  return <PlaceholderScreen title={t.tabs.qibla} note={t.placeholder.qibla} />;
}

```

Стъпка 12

Стартиране на телефона

Проектът ползва собствен development build, а не Expo Go. Build-ът се прави веднъж. После при промени в кода hot reload работи през Metro, както си свикнал.

Важно за Windows: native build-ът (CMake/ninja) генерира пътища с дължина около 300 знака. Старият ninja 1.10, който идва с CMake 3.22 в Android SDK, не поддържа пътища над 260 знака и build-ът спира с `Filename longer than 260 characters`. Краткият път до проекта сам по себе си **не стига**. Нужни са три неща:

- **Дълги пътища в Windows.** PowerShell като администратор (командата е по-долу), после рестарт на компютъра.
- **CMake 3.31.6** (с нов ninja): Android Studio → Settings → Languages & Frameworks → Android SDK → SDK Tools → отметни „Show Package Details“ → CMake → 3.31.6 → Apply. Проектът вече го изисква чрез `expo-build-properties` в `app.json`.
- **Чист build:** изтрий папката `android`, за да се генерира наново с новата настройка.

PowerShell като администратор, след това рестарт

```
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem" -Name "LongPathsEnabled" -Value 1 -PropertyType DWORD -Force
```

Чист build след инсталирането на CMake 3.31.6

```
cd C:\dev\ezan
Remove-Item -Recurse -Force android
npm expo run:android
```

Препоръчително е и пътят до проекта да е кратък и без интервали (например `C:\dev\ezan`), но това е допълнителна мярка, не решението.

Вариант А – локално (Android Studio е инсталирано, телефонът е вързан с USB):

= `npm expo run:android`. Генерира папката `android/`, build-ва и инсталира приложението

```
npm run android
```

Вариант Б – в облака с EAS (без Android Studio):

След 10–15 мин. получаваш линк/QR код за APK-то

```
npm eas-cli@latest login
npm eas-cli@latest init
npm eas-cli@latest build --profile development --platform android
```

Инсталирай APK-то на телефона и стартирай сървъра за разработка:

Отвори приложението „Езан“ на телефона и то се свързва с компютъра

```
npm expo start
```

Бърз преглед в браузъра, без телефон:

Терминал

```
npm expo start --web
```

- Папките `android/` и `ios/` се **генерират** (Continuous Native Generation) и са в `.gitignore`. Не ги редактирай на ръка. Native промени се правят през `app.json` и config plugins.
- Нов build е нужен само когато добавиш библиотека с native код. За промени в TypeScript е достатъчен hot reload.

Стъпка 13

Проверка

Преди всеки commit:

Конфигурацията на ESLint е в `eslint.config.js`

```
npm run typecheck  
npm run lint
```

Какво трябва да видиш на телефона:

- Фонът е в цвета на текущата молитва, а шарката се вижда едва-едва.
- Обратното броене се сменя всяка секунда, без цифрите да „подскачат“.
- Текущата молитва е в злато с пил „СЕГА“, миналите са приглушени.
- Тап на камбанка сменя режима и показва toast. При Изгрев вариант „езан“ няма.
- След рестарт на приложението режимите на камбанките са запазени.
- Когато настъпи следващата молитва, фонът плавно сменя цвета си.
- Смени езика на телефона на английски (или друг) – приложението веднага минава на английски. Обратно на български – на български.
- Android 13+: Настройки → Приложения → Adhan/Езан → Език. Там може да се избере език само за приложението.
- Името под иконата е „Езан“ на телефон на български и „Adhan“ на всеки друг език.

Изчислените часове (Диянет, Аср по Шафии) за София на 28 септември 2026 са: Фаджр 05:46, Изгрев 07:13, Зухр 13:22, Аср 16:40, Магриб 19:21, Иша 20:42. Ако календарът на Мюфтийството се разминава, това е първата задача за етап 2.